

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 2
з дисципліни «Основи програмування»
«Графічна мова програмування Scratch»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття: функцій, умов, булевих виразів, циклів.

систем числення та навчитися переводити числа з однієї системи в іншу

Обладнання та програмне забезпечення

- IBM-сумісна персональна обчислювальна машина;
- графічна мова програмування Scratch (<https://scratch.mit.edu/>).

1. Теоретичні відомості

1.1. Системи числення

Для переведення числа N з десяткової системи в систему з основою b використовується метод послідовного ділення на b із запам'ятовуванням залишків. Алгоритм:

- Обчислити остачу $r = N \bmod b$.
- Записати r (якщо $r \geq 10$, замінити на відповідну літеру).
- Замінити N на $N // b$ (цілочисельне ділення).
- Повторювати кроки 1–3, поки $N \neq 0$.
- Залишки, прочитані у зворотному порядку, утворюють результат.

У шістнадцятковій системі ($b = 16$) цифри 10–15 позначаються літерами A–F.

1.2. Рівняння руху по параболі

Розглянемо матеріальну точку, якій надано початкову швидкість v_0 під кутом α до горизонту. На точку діє лише сила тяжіння (опором повітря нехтуємо). Прискорення за віссю x дорівнює нулю, за віссю y — $g = -9.8 \text{ м/с}^2$.

Розкладемо початкову швидкість на компоненти:

$$v_x = v_0 \cdot \cos(\alpha)$$

$$v_y = v_0 \cdot \sin(\alpha)$$

Оскільки рух по горизонталі рівномірний, а по вертикалі — рівноприскорений, координати в момент часу t мають вигляд:

$$x(t) = x_0 + v_x \cdot t$$

$$y(t) = y_0 + v_y \cdot t + (g \cdot t^2) / 2$$

Підставивши компоненти швидкості, отримуємо повні формули:

$$x(t) = x_0 + v_0 \cdot \cos(\alpha) \cdot t$$

$$y(t) = y_0 + v_0 \cdot \sin(\alpha) \cdot t + (g \cdot t^2) / 2$$

У середовищі Scratch система координат має початок у центрі сцени ($x \in [-240; 240]$, $y \in [-180; 180]$). Щоб фізичні одиниці коректно відображалися в пікселях екрана, вводять масштабний коефіцієнт M . Тоді формули набувають вигляду:

$$x(t) = x_0 + v_0 \cdot \cos(\alpha) \cdot t \cdot M$$

$$y(t) = y_0 + v_0 \cdot \sin(\alpha) \cdot t \cdot M + (g \cdot t^2) / 2 \cdot M$$

У програмі вибрано: $\alpha = 45^\circ$, $v_0 = 10$, $g = -9.8$, $M = 42.7$, $x_0 = -220$, $y_0 = -140$, крок інтегрування $dt = 0.01$.

2. Опис створених програм

2.1. Переведення десяткового числа у двійкову систему

Програма запитує десяткове число, зберігає його у змінну «число», обнуляє результат та остачу. У циклі «repeat until число = 0» виконується така послідовність:

- остача $\leftarrow$ число mod 2;
- результат $\leftarrow$ join(остача, результат);
- число $\leftarrow$ (число – остача) / 2.

Цикл триває, поки число не стане нулем. Залишки додаються на початок рядка, тому результат одразу формується у правильному порядку (від старшого розряду до молодшого).

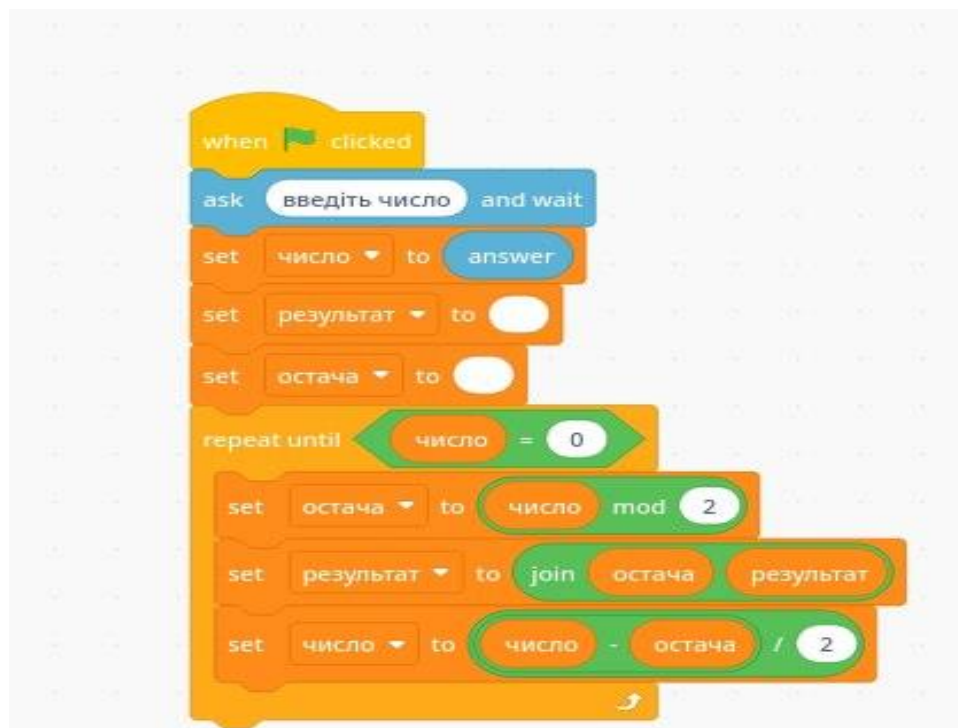
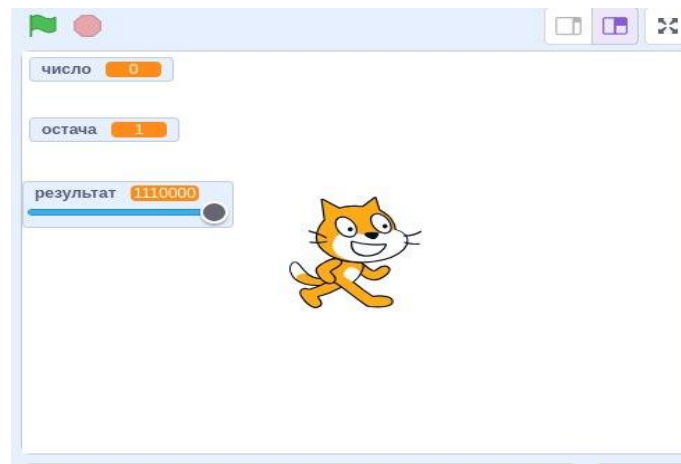


Рис. 1. Скрипт програми переведення у двійкову систему

*Рис. 2. Введення числа 112**Рис. 3. Результат: 1110000₂*

2.2. Переведення десяткового числа у шістнадцяткову систему

Алгоритм аналогічний двійковому, але основа $b = 16$. Після обчислення остачі (число $\bmod 16$) виконується перевірка: якщо остача ≥ 10 , її замінюють на літеру за правилом $10 \rightarrow A$, $11 \rightarrow B$, $12 \rightarrow C$, $13 \rightarrow D$, $14 \rightarrow E$, $15 \rightarrow F$. Далі остача (або літера) додається на початок результату, а число зменшується

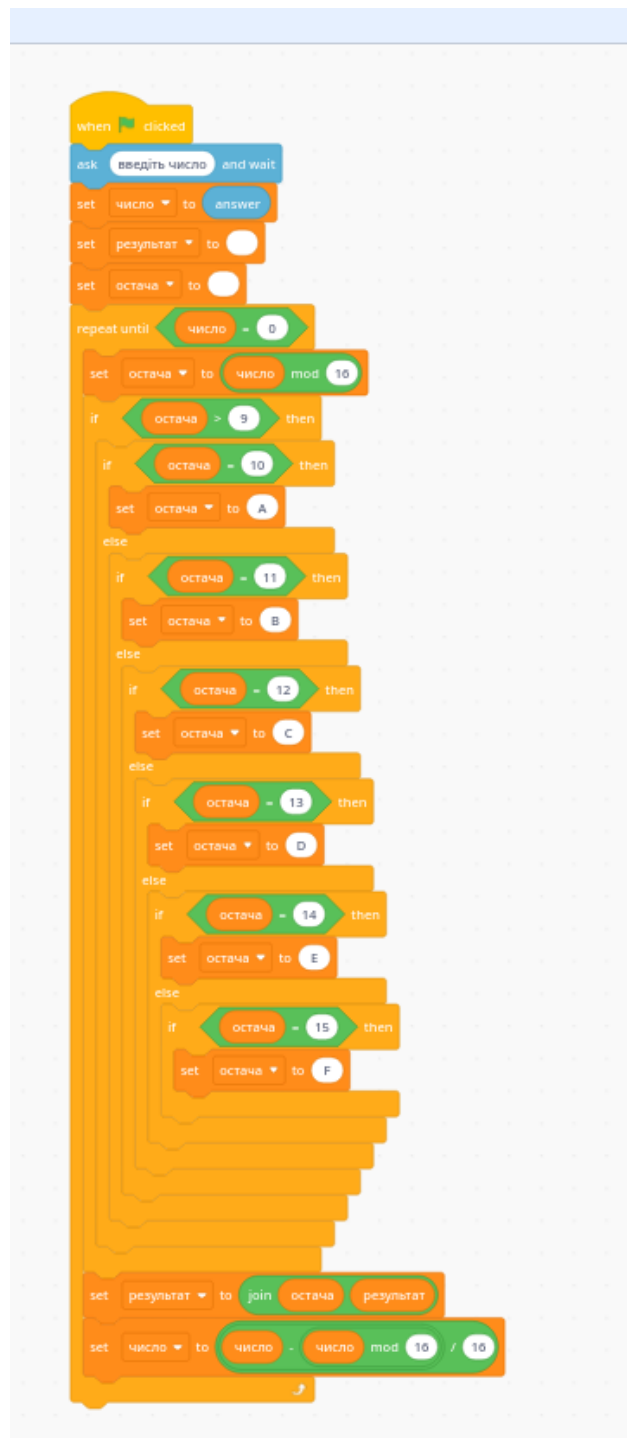


Рис. 4. Скрипт програми переведення у шістнадцяткову систему

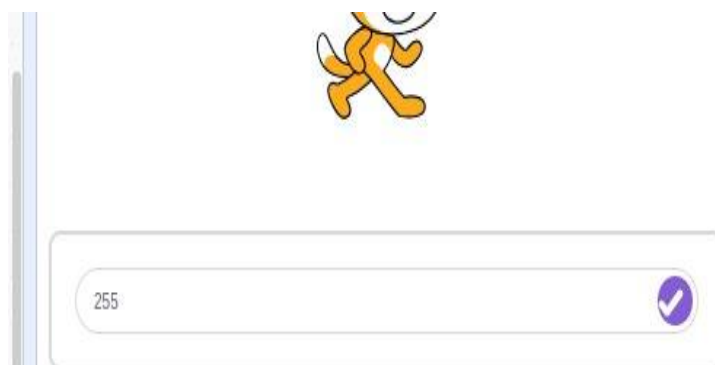
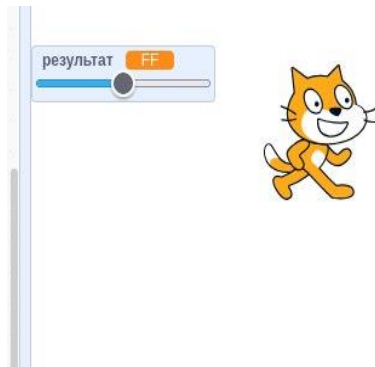


Рис. 5. Введення числа 255

Рис. 6. Результат: FF_{16}

2.3. Рух спрайту по параболі під дією сили тяжіння

Програма імітує фізичний рух матеріальної точки. Початкові умови та параметри:

$$x_0 = -220, y_0 = -140 \text{ (лівий нижній кут сцени)}$$

$$v_0 = 10, \alpha = 45^\circ, g = -9.8, M = 42.7, dt = 0.01$$

Компоненти початкової швидкості обчислюються один раз перед циклом:

$$v_x = v_0 \cdot \cos(45^\circ) \approx 10 \cdot 0.707 = 7.07$$

$$v_y = v_0 \cdot \sin(45^\circ) \approx 10 \cdot 0.707 = 7.07$$

Внутрішній цикл «repeat until ($y > y_0$) or ($y < y_0$)» інтегрує рівняння руху з кроком dt , поки спрайт не повернеться на рівень «землі» (y_0). На кожній ітерації:

$$t = t + dt$$

$$x = x_0 + v_x \cdot t \cdot M$$

$$y = y_0 + v_y \cdot t \cdot M + (g \cdot t^2) / 2 \cdot M$$

$$\text{go to x: } x \text{ y: } y$$

Оскільки $g < 0$, доданок $(g \cdot t^2)/2$ з часом зменшує y , формуючи параболічну траєкторію. Умова виходу з циклу спрацьовує, коли спрайт перетинає горизонталь $y = y_0$, тобто «приземляється».

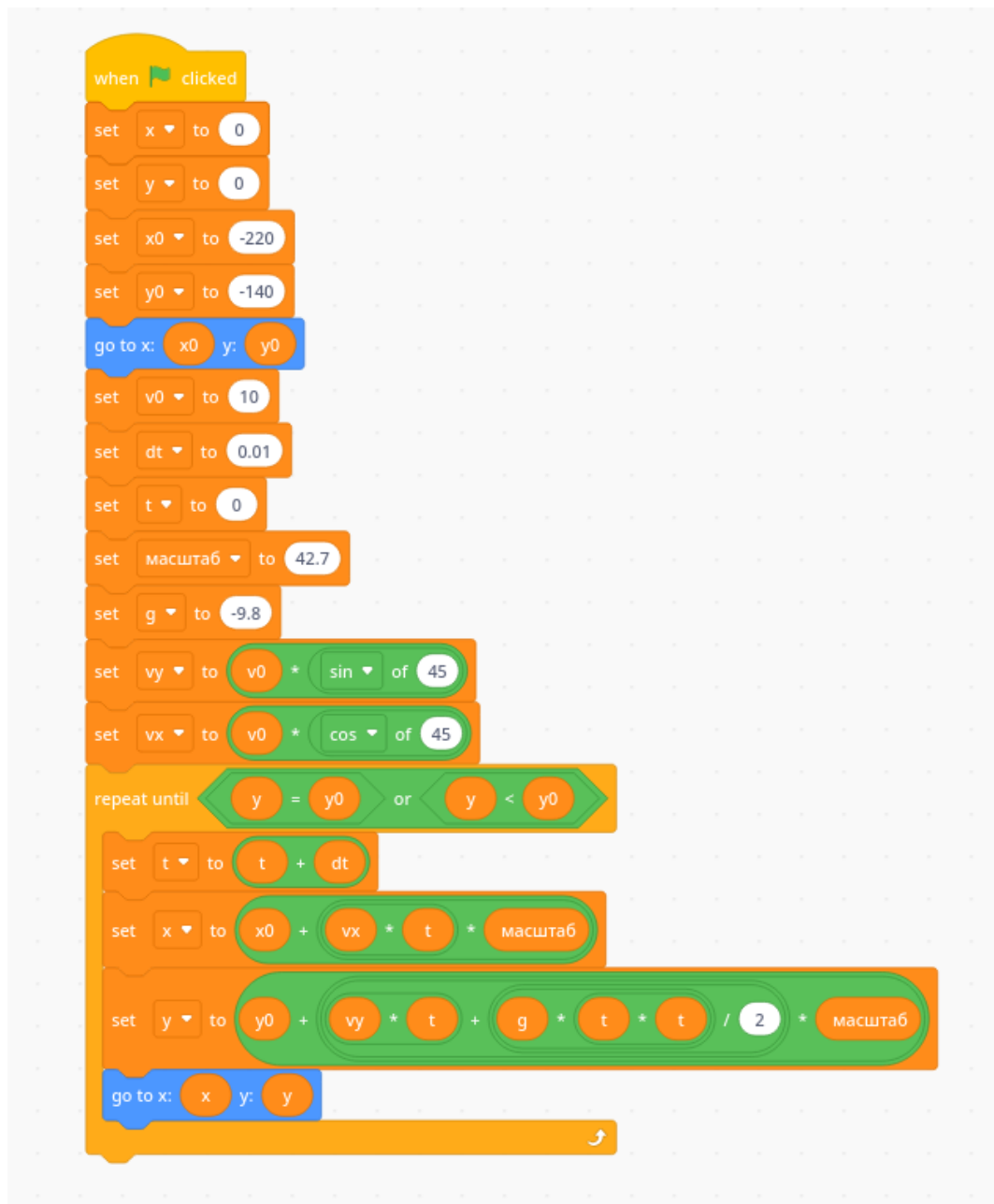


Рис. 7. Скрипт програми руху котика по параболі

Аналіз параметрів. Масштаб $M = 42.7$ підбрано експериментально так, щоб дальність польоту (приблизно $v_0^2 \cdot \sin(2\alpha) / |g| \cdot M$) відповідала ширині робочої області Scratch (близько 440 пікселів). Крок $dt = 0.01$ забезпечує достатньо плавну анімацію: занадто великий крок робить траєкторію стрибкоподібною, занадто малий — сповільнює виконання. Початкові координати $(-220, -140)$ розміщують спрайт біля лівого нижнього кута, залишаючи запас по вертикалі для підйому.

3. Висновки

У ході виконання лабораторної роботи вивчено основні конструкції графічної мови програмування Scratch.

Найцікавішою та найскладнішою виявилася програма руху по параболі.

Основні труднощі:

- підбір початкових координат і масштабу M , щоб траєкторія виглядала природно і не виходила за межі сцени;
- правильне формулювання умови виходу з циклу (порівняння поточної y з початковою y_0), щоб кіт приземлився і не провалився під «землю»;
- балансування кроку часу dt — занадто великий крок робив рух стрибкоподібним, занадто малий — сповільнював анімацію;
- врахування того, що в Scratch початок координат — у центрі екрана, а не в лівому нижньому куті, тому формули кінематики потрібно було масштабувати.

Незважаючи на ці нюанси, усі три програми працюють коректно і відповідають математичним моделям. Робота дозволила зрозуміти принципи побудови алгоритмів із використанням циклів і умов у візуальному середовищі, а також практично застосувати формули рівномірного та рівноприскореного руху.